

**INTEGRASI SISTEM AKSELERATOR CNN BERBASIS
KOMPUTASI FIXED-POINT 16-BIT PADA BOARD FPGA**

SKRIPSI



Disusun oleh:
ALEX CINATRA HUTASOIT
22/505820/TK/55377

**JURUSAN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

HALAMAN PENGESAHAN

INTEGRASI SISTEM AKSELERATOR CNN BERBASIS KOMPUTASI FIXED-POINT 16-BIT PADA BOARD FPGA

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik Program S-1
Pada Jurusan Teknik Elektro dan Teknologi Informasi Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Alex Cinatra Hutasoit
22/505820/TK/55377

Telah disetujui dan disahkan
pada tanggal 15 Juni 2026

Dosen Pembimbing I

Dosen Pembimbing II

Igi Ardiyanto, S.T., M.Eng.
NIP. 1987 0109 2020 12 1 005

Agus Bejo, S.T., M.Eng.
NIP. 1980 0101 2015 04 1 002

HALAMAN PERSEMBAHAN

*Untuk Ibu, Bapak,
dan Adik-adikku tercinta.*

KATA PENGANTAR

Assalamu'alaikum Wr. Wb.

Puji dan syukur ke hadirat Allah Yang Mahakuasa atas berkat dan karunia Roh Kudus sehingga tugas akhir ini dapat terselesaikan dengan baik. Penulis mengucapkan terima kasih atas arahan, bantuan, dukungan, dan doa yang telah diberikan oleh berbagai pihak. Pada kesempatan ini penulis mengucapkan terima kasih sedalam-dalamnya kepada pihak-pihak, di antaranya:

1. Bapak Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE., selaku Ketua Jurusan Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada,
2. Bapak Dr.Eng. Ir. Igi Ardiyanto, S.T., M.Eng., IPM., ASEAN Eng., SMIEEE., selaku dosen pembimbing pertama yang telah memberikan banyak bantuan, bimbingan, serta arahan dalam Tugas Akhir ini,
3. Bapak Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM., selaku dosen pembimbing kedua yang juga telah memberikan banyak bantuan, bimbingan, serta arahan dalam Tugas Akhir dan kegiatan-kegiatan yang lain,
4. Bapak Widyawan, S.T., M.Sc., Ph.D., selaku dosen pembimbing akademik yang telah memberikan pendampingan dan arahan selama masa perkuliahan,
5. Seluruh Dosen di Jurusan Teknik Elektro dan Teknologi Informasi FT UGM, yang tidak bisa disebutkan satu-satu, atas ilmu dan bimbingannya selama penulis berkuliah di JTETI,

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat dalam bidang teknologi informasi untuk penelitian selanjutnya di masa mendatang.

Wassalamu'alaikum Wr. Wb.

Yogyakarta, 15 Juni 2026

Alex Cinatra Hutasoit

DAFTAR ISI

HALAMAN PENGESAHAN	iii
HALAMAN PERSEMBAHAN	iii
KATA PENGANTAR	iv
DAFTAR ISI	vi
DAFTAR TABEL	vii
DAFTAR GAMBAR	viii
DAFTAR SINGKATAN	ix
Intisari	xii
<i>Abstract</i>	xiii
I LATAR BELAKANG	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	2
1.4 Batasan Masalah	2
1.5 Manfaat Penelitian	3
1.6 Sistematika Penulisan	3
II TINJAUAN PUSTAKA DAN DASAR TEORI	4
2.1 Tinjauan Pustaka	4
2.2 Landasan Teori	6
2.2.1 Backpressure	6
2.2.2 Finite State Machine	7
2.2.3 Multiply Accumulate	8
2.2.4 Time Multiplexer	9
2.2.5 Shift Register	9
2.2.6 First In First Out (FIFO)	11

2.2.7	Kuantitasi	12
2.2.8	Double Data Rate (DDR)	14
2.2.9	FPGA ARTIX 7	15
2.2.10	OV7670	16
2.2.11	VGA	17
2.3	Analisis Perbandingan Metode	18
DAFTAR PUSTAKA		24

DAFTAR TABEL

Tabel 2.1	Spesifikasi utama Arty A7-100T	16
Tabel 2.2	Perbandingan implementasi sistem pada penelitian terkait . .	18
Tabel 2.3	Perbandingan performa penelitian terkait	18

DAFTAR GAMBAR

Gambar 2.1	Aliran sinyal <i>valid</i> dan <i>ready</i> pada sistem pipeline	6
Gambar 2.2	Ilustrasi data <i>input shift register</i>	10
Gambar 2.3	Ilustrasi FIFO	11
Gambar 2.4	Ilustrasi hubungan AXI, MIG, dan DDR pada sistem pemindahan data berbasis memori eksternal.	14

DAFTAR SINGKATAN

A

AXI Advanced eXtensible Interface

B

BD Block Design

BRAM Block Random Access Memory

BUFG Global Clock Buffer

C

CLK Clock

CNN Convolutional Neural Network

CONV Convolution

CPU Central Processing Unit

CSV Comma-Separated Values

D

DRC Design Rule Check

DSP Digital Signal Processing

DDR Double Data Rate

F

FF Flip Flop

FIFO First In First Out

FWFT First Word Fall Through

FPGA Field Programmable Gate Array

FPS Frame Per Second

G

GPIO General-Purpose Input/Output

GPU Graphics Processing Unit

H

HDL Hardware Descriptive Language

I

I/O Input Output

I2C Inter-Integrated Circuit

IOB Input/Output Block

J

JSON Javascript Object Notation

L

LUT Look-Up Table

M

MIG Memory Interface Generator

MNIST Modified National Institute of Standards and Technology

MUX Multiplexer

N

NPZ NumPy Compressed Archive

S

SCCB Serial Camera Control Bus

SOC System On Chip

SPI Serial Peripheral Interface

T

TNS Total Negative Slack

THS Total Hold Slack

U

UART Universal Asynchronous Receiver-Transmitter

UI User Interface

URAM Ultra Random Access Memory

W

WNS Worst Negative Slack

WHS Worst Hold Slack

WPWS Worst Pulse Width Slack

Intisari

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt.

Kata kunci : *CNN, ARTY 7, OV7670, Serial, Komputasi Pararel.*

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt.

Keywords : CNN, ARTY 7, OV7670, Serial, Pararel Computation.

BAB I

LATAR BELAKANG

1.1 Latar Belakang Masalah

Kemampuan CNN dalam melakukan ekstraksi fitur hirarkis pada spatial data matriks menjadikannya sebagai salah satu metode yang paling banyak digunakan dalam proses pengolahan citra dan visi. Proses ekstraksi fitur pada data gambar berskala besar dilakukan oleh secara otomatis dan menjadikan CNN sebagai fondasi utama dalam implementasi pengolahan citra dan visi untuk pengenalan pola, objek hingga karakter [1, 2]. Keunggulan ini mendorong adopsi dan implementasi di berbagai bidang terutama sistem tertanam. Saat ini implementasi CNN banyak dilakukan di GPU dan CPU. Konsumsi daya pada GPU sangat besar dan boros baik pada saat proses pelatihan model maupun pada saat mode inferensial[3, 4]. Hal ini terjadi terutama pada GPU, karena overhead komunikasi data bus pada batch berukuran kecil sehingga penggunaan daya sangat tidak efisien.

Keterbatasan implementasi CNN pada CPU dan GPU menjadi semakin penting ketika CNN diterapkan pada sistem tertanam yang membutuhkan pemrosesan citra secara langsung, cepat, dan hemat daya[5]. Pada aplikasi seperti kamera cerdas, robotika, dan kendaraan otonom, data visual diperoleh secara terus-menerus dari sensor sehingga proses komputasi tidak cukup hanya akurat, tetapi juga harus memiliki latensi rendah. Oleh karena itu, pendekatan *edge computation* dan *embedded vision* menjadi salah satu solusi karena proses inferensi dapat dilakukan lebih dekat dengan sumber data.

Salah satu alternatif perangkat keras yang dapat digunakan untuk mempercepat komputasi CNN adalah FPGA. FPGA memiliki keunggulan karena arsitektur perangkat kerasnya dapat dikonfigurasi sesuai kebutuhan algoritma. Berbeda dengan CPU dan GPU yang menggunakan arsitektur komputasi umum, FPGA memungkinkan perancang untuk membangun jalur data khusus, menerapkan paralelisme pada tingkat operasi, serta menggunakan teknik *pipeline* untuk meningkatkan *throughput*. Karakteristik ini sesuai dengan operasi CNN yang bersifat berulang, terstruktur, dan memiliki pola komputasi yang dapat dipetakan ke dalam perangkat keras pola [6, 7].

Penelitian ini berfokus pada perancangan dan implementasi akselerator CNN berbasis FPGA dengan komputasi *fixed-point* yang berjalan dengan daya rendah. Ak-

selerator dirancang untuk menjalankan proses inferensi CNN secara perangkat keras dengan memanfaatkan paralelisme, *pipeline*, dan optimasi representasi data. Selain itu, penelitian ini juga membahas integrasi sistem dengan perangkat kamera dan jalur tampilan citra, sehingga sistem tidak hanya diuji sebagai modul komputasi terpisah, tetapi juga sebagai bagian dari sistem *embedded vision* yang menerima data visual dari lingkungan nyata.

1.2 Rumusan Masalah

Implementasi CNN pada CPU dan GPU masih membutuhkan *resource* komputasi yang besar, terutama dari sisi konsumsi daya dan efisiensi inferensi pada sistem tertanam. Upaya untuk menekan konsumsi daya pada perangkat komputasi umum sering kali berdampak pada penurunan performa sehingga latensi inferensi menjadi lebih besar. Sedangkan kebutuhan akan sistem yang efisien secara daya dan instan semakin dibutuhkan.

1.3 Tujuan Penelitian

Tujuan penelitian ini adalah untuk menunjukkan potensi FPGA sebagai platform ideal disamping CPU dan GPU pada implementasi CNN sampai ketahap *Deployment*.

1.4 Batasan Masalah

Batasan masalah pada penelitian ini adalah:

1. Pada penelitian ini *board* FPGA yang digunakan adalah ARTY 7 100T.
2. Kamera yang digunakan adalah OV7670.
3. Streaming video dilakukan lewat VGA, dengan modul VGA PMOD Digilent.
4. Keterbatasan *resource* pada FPGA memungkinkan penelitian menggunakan model yang tidak terlalu besar.
5. *Dataset* yang digunakan adalah *Dataset* MNIST, dengan ukuran 28x28 *pixel channel* 1.
6. *Training* model dilakukan di CPU python.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan implementasi CNN pada perangkat FPGA, khususnya pada penggunaan komputasi *fixed-point*, pemetaan paralelisme operasi konvolusi, dan penerapan *pipeline* untuk mempercepat proses inferensi. Selain itu, penelitian ini dapat menjadi referensi dalam memahami proses perancangan akselerator CNN berbasis perangkat keras untuk aplikasi *embedded vision*.

1.6 Sistematika Penulisan

BAB I : PENDAHULUAN

Pada bab ini dijelaskan latar belakang, rumusan masalah, batasan, tujuan, manfaat, dan sistematika penulisan.

BAB II : TINJAUAN PUSTAKA DAN LANDASAN TEORI

Pada bab ini dijelaskan teori-teori dan penelitian terdahulu yang digunakan sebagai acuan dan dasar dalam penelitian.

BAB III : METODOLOGI PENELITIAN

Pada bab ini dijelaskan metode yang digunakan dalam penelitian meliputi langkah kerja, pertanyaan penelitian, alat dan bahan, serta tahapan dan alur penelitian.

BAB IV : HASIL DAN PEMBAHASAN

Pada bab ini dijelaskan hasil penelitian dan pembahasannya.

BAB V : KESIMPULAN DAN SARAN

Pada bab ini ditulis kesimpulan akhir dari penelitian dan saran untuk pengembangan penelitian selanjutnya.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Penelitian Che et al. dan Qasaimeh et al. menunjukkan bahwa pemilihan platform komputasi untuk aplikasi visi sangat dipengaruhi oleh karakteristik algoritma, latensi, dan kebutuhan energi [8, 9]. CPU memiliki fleksibilitas tinggi, tetapi kurang efisien untuk komputasi CNN yang membutuhkan paralelisme besar. GPU mampu memberikan throughput tinggi, namun umumnya membutuhkan daya dan bandwidth memori yang lebih besar. Dalam konteks tersebut, FPGA menjadi alternatif karena operasi CNN seperti konvolusi, *multiply-accumulate*, *buffering*, dan *pipeline* dapat dipetakan langsung ke sumber daya perangkat keras seperti DSP, BRAM, LUT, dan FF, sehingga sesuai untuk sistem *embedded vision* yang membutuhkan latensi rendah dan konsumsi daya efisien [10].

Berdasarkan penelitian Solovyev et al., implementasi CNN pada FPGA untuk *real-time processing* dilakukan dengan mengubah representasi *floating-point* menjadi *fixed-point* agar lebih sesuai dengan komputasi perangkat keras [11]. Penelitian ini menggunakan dataset MNIST dengan augmentasi berupa rotasi acak 10 derajat, inversi warna, serta penambahan *noise* dari 0% sampai 10%. Setelah proses pelatihan, parameter CNN disimpan ke dalam blok memori FPGA dan digunakan pada proses inferensi. Implementasi dilakukan pada board *DE0-Nano* berbasis FPGA *Intel Cyclone IV*, dengan masukan citra dari kamera OV7670 dan keluaran tampilan melalui VGA. Citra dari kamera direduksi menjadi *grayscale* berukuran 28×28 piksel untuk diproses oleh CNN, sehingga sistem mampu berjalan secara *real-time* dengan kecepatan lebih dari 150 *frame per second*. Penelitian ini menghasilkan akurasi model yang tidak berubah dengan model training sebelum kuantisasi.

Pada sisi kamera, penelitian Solovyev et al. juga memperlihatkan penggunaan OV7670 sebagai sumber citra untuk sistem CNN berbasis FPGA [11]. Konfigurasi kamera OV7670 dilakukan melalui *Serial Camera Control Bus* atau SCCB untuk mengatur format keluaran citra, resolusi, dan mode operasi sensor [12]. Data piksel kemudian dikirim melalui *pixel clock*, bus data paralel, serta sinyal sinkronisasi *HREF* dan *VSYNC*. Sinyal *HREF* menandai data piksel valid pada satu baris, sedangkan *VSYNC* menandai pergantian *frame*. Sistem inferensi *end-to-end* CNN

berbasis kamera tidak hanya membutuhkan akselerator inferensi, tetapi juga akuisisi citra yang sinkron terhadap tampilan VGA.

Penelitian Qiu et al. mengimplementasikan CNN pada FPGA *Xilinx ZC706* dengan model *VGG16-SVD* untuk klasifikasi citra ImageNet menggunakan representasi 16-bit *fixed-point* [13]. Hasil implementasi menunjukkan akurasi *top-5* sebesar 86.66%, performa inferensi sebesar 4.45 FPS, *throughput* sekitar 137 GOPS untuk keseluruhan jaringan, konsumsi daya sebesar 9.63 W. Penelitian ini menunjukkan bahwa kuantisasi 16-bit *fixed-point* masih dapat mempertahankan akurasi yang layak pada model CNN berskala besar. Dari sisi arsitektur, operasi konvolusi menjadi bagian komputasi dominan sehingga pemetaan operasi *multiply-accumulate* ke DSP, penggunaan *buffer*, serta pengaturan akses memori menjadi faktor penting dalam meningkatkan *throughput*.

Penelitian Dua et al. melalui arsitektur *Systolic-CNN* menunjukkan implementasi CNN berbasis *OpenCL* pada FPGA dengan pendekatan yang lebih fleksibel terhadap banyak model [14]. Berbeda dari Solovyev et al. dan Qiu et al. yang menggunakan *fixed-point*, penelitian ini menggunakan format 32-bit *floating-point*. Model yang diuji meliputi *AlexNet*, *ResNet-50*, *ResNet-152*, *RetinaNet*, dan *Light-weight RetinaNet*. Implementasi dilakukan pada board *Intel Arria 10 GX1150* dan *Intel Stratix 10 GX2800*. Pada *Arria 10*, inferensi *AlexNet* membutuhkan waktu sekitar 7 ms dan *ResNet-50* sekitar 84 ms, sedangkan pada *Stratix 10* latensinya menjadi sekitar 2 ms dan 33 ms. Paper ini tidak melaporkan konsumsi daya maupun efisiensi energi dalam GOPS/W, sehingga lebih tepat digunakan sebagai referensi arsitektur *systolic array*, pemanfaatan DSP, dan fleksibilitas *OpenCL*.

Selain penelitian CNN, integrasi kamera dan tampilan juga ditunjukkan oleh Navaneethan et al. melalui implementasi *video streaming* dari kamera OV7670 ke VGA pada board FPGA *Spartan-6* dan *Artix-7* menggunakan Verilog dan Xilinx Vivado [15]. Penelitian ini tidak berfokus pada inferensi CNN, tetapi menunjukkan bahwa jalur akuisisi kamera, sinkronisasi piksel, dan tampilan VGA dapat diimplementasikan dengan penggunaan sumber daya yang relatif kecil, yaitu 530 *slice registers*. Hasil tersebut relevan sebagai pembandingan bagian antarmuka kamera dan display, terutama karena sistem CNN berbasis kamera tetap membutuhkan jalur video yang stabil sebelum citra diproses oleh akselerator.

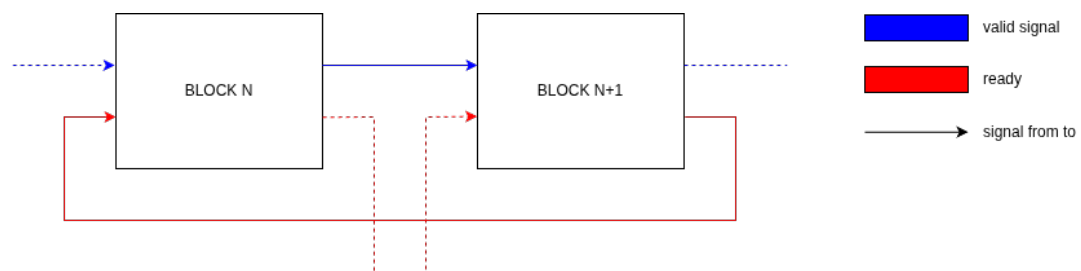
Pada sistem yang memproses citra berukuran besar, penggunaan *frame buffer* menjadi penting karena data kamera dan data tampilan bekerja pada aliran waktu yang berbeda. Untuk resolusi VGA 640×480 dengan format RGB, kapasitas BRAM

pada FPGA umumnya tidak cukup untuk menyimpan satu atau dua *frame* penuh [16]. Oleh karena itu, pada rancangan ini DDR digunakan sebagai *frame buffer*, sedangkan akses baca dan tulis dikendalikan melalui antarmuka AXI menuju MIG. AXI bertindak sebagai jalur transaksi antara modul *master* dan MIG sebagai pengendali DDR, dan proses akses memori dilakukan setelah MIG selesai dikalibrasi [17, 18].

2.2 Landasan Teori

2.2.1 Backpressure

Backpressure adalah mekanisme pengendalian aliran data yang digunakan ketika suatu blok belum siap menerima data dari blok sebelumnya. Mekanisme ini umumnya bekerja dengan sinyal *valid* dan *ready*. Sinyal *valid* menunjukkan bahwa data dari pengirim sudah tersedia, sedangkan sinyal *ready* menunjukkan bahwa penerima siap mengambil data tersebut [19, 20].



Gambar 2.1: Aliran sinyal *valid* dan *ready* pada sistem pipeline

Berdasarkan Gambar 2.1, sinyal *valid* mengalir searah dengan aliran data, yaitu dari blok proses sebelumnya menuju blok proses berikutnya. Sebagai contoh, ketika blok proses ke- N menghasilkan data, blok tersebut akan mengaktifkan sinyal *valid* ke blok proses ke- $N+1$. Sebaliknya, sinyal *ready* mengalir ke arah belakang, yaitu dari blok penerima menuju blok pengirim. Sinyal ini digunakan untuk memberitahu bahwa blok penerima sudah siap menerima dan memproses data berikutnya.

Jika sinyal *ready* belum aktif, maka pengirim harus menahan data sampai penerima siap. Dengan cara ini, data tidak hilang atau tertimpa oleh data berikutnya. Pada sistem berbasis *pipeline*, konsep backpressure penting untuk menjaga aliran data tetap teratur, terutama ketika setiap blok memiliki waktu proses yang berbeda [19].

2.2.2 Finite State Machine

Finite State Machine atau FSM merupakan model kendali sekuensial yang merepresentasikan perilaku sistem dalam sejumlah *state* terbatas. Pada setiap siklus kerja, FSM membaca *input*, mengevaluasi kondisi transisi, menentukan *state* berikutnya, kemudian menghasilkan *output* sesuai dengan aturan transisi yang telah ditentukan seperti pada *Pseudocode* 1. Dengan pendekatan ini, proses kerja yang kompleks dapat dibagi menjadi beberapa tahap yang lebih terstruktur, sehingga alur kendali sistem menjadi lebih mudah dirancang, diamati, dan diverifikasi.

Algorithm 1 Finite State Machine Algorithm

Require: inputSignal, currentState, TransitionSet

Ensure: updated currentState, outputSignal

```

1: procedure UPDATEFSM(inputSignal)                                ▷ Initialize default values
2:   nextState ← currentState
3:   outputSignal ← DEFAULT_OUTPUT
                                     ▷ Evaluate all possible state transitions
4:   for all transition ∈ TransitionSet do
5:     if transition.sourceState == currentState and transition.inputCondition(inputSignal) == true then
                                     ▷ State transition
6:       nextState ← transition.targetState
                                     ▷ Generate output
7:       outputSignal ← transition.outputAction(currentState, inputSignal,
nextState)
8:       break
9:     end if
10:  end for
                                     ▷ Update the current state
11:  currentState ← nextState
                                     ▷ Return the generated output
12:  return outputSignal
13: end procedure

```

Dalam sistem digital, FSM banyak digunakan untuk mengatur proses yang membutuhkan urutan kerja tertentu, seperti proses inisialisasi, pembacaan data, penulisan data, sinkronisasi sinyal, dan pengendalian jalur *data path*. Pada penelitian ini, FSM digunakan sebagai bagian penting dalam pengendalian proses *input-output*, kendali transaksi AXI, kendali kamera, serta pengaturan tahapan komputasi pada CNN *Accelerator*. Kebutuhan kendali tersebut menjadi semakin penting pada implementasi

CNN di FPGA, karena proses komputasi CNN terdiri dari beberapa operasi berurutan seperti konvolusi, aktivasi, *pooling*, dan klasifikasi. CNN sendiri banyak digunakan pada tugas klasifikasi citra karena kemampuannya dalam mengekstraksi fitur secara bertingkat [2, 1]. Oleh karena itu, FSM berperan sebagai pengatur aliran proses agar setiap modul dapat bekerja secara sinkron, mulai dari penerimaan data citra, pemrosesan fitur, hingga keluaran hasil klasifikasi.

Pada sistem yang lebih kompleks, transisi FSM tidak hanya ditentukan oleh *input* utama, tetapi juga oleh kondisi kesiapan antarblok. Mekanisme ini berkaitan dengan *backpressure*, yaitu kondisi ketika sebuah modul harus menahan proses karena modul berikutnya belum siap menerima data. Dalam implementasi *advanced backpressure*, FSM dapat diperluas dengan sinyal kendali seperti *valid*, *ready*, *busy*, *done*, *fifo empty*, dan *fifo full*. Sinyal-sinyal tersebut digunakan sebagai syarat transisi agar perpindahan *state* hanya terjadi ketika data tersedia dan jalur keluaran siap menerima data. Dengan demikian, FSM tidak hanya berfungsi sebagai pengatur urutan kerja, tetapi juga sebagai mekanisme kontrol aliran data untuk mencegah kehilangan data, pembacaan data kosong, atau penulisan data ketika buffer sudah penuh.

2.2.3 Multiply Accumulate

Multiply Accumulate (MAC) adalah operasi komputasi yang terdiri dari proses perkalian dua nilai dan penjumlahan hasilnya ke nilai akumulasi sebelumnya. Secara umum, operasi MAC dapat dituliskan sebagai berikut:

$$y = \sum_{i=0}^{N-1} x_i w_i + b \quad (2.1)$$

Pada persamaan 2.1, x_i merupakan data masukan, w_i merupakan bobot, dan b merupakan nilai bias. Operasi ini banyak digunakan pada komputasi (CNN), terutama pada proses konvolusi dan *fully connected layer*. Pada proses tersebut, setiap nilai keluaran diperoleh dari hasil perkalian antara data masukan dan bobot, kemudian seluruh hasil perkalian tersebut dijumlahkan.

Pada FPGA, operasi MAC dapat diimplementasikan menggunakan blok logika umum atau menggunakan sumber daya khusus berupa DSP. Pada FPGA seri 7, blok DSP48E1 memiliki struktur yang mendukung operasi perkalian, penjumlahan, dan akumulasi, sehingga sesuai digunakan untuk mempercepat operasi MAC [21]. Penggunaan DSP membuat operasi MAC lebih efisien dibandingkan jika seluruh ope-

rasi aritmetika dibangun hanya menggunakan LUT dan FF.

Namun, jumlah DSP pada FPGA bersifat terbatas. Jika setiap operasi perkalian dan akumulasi pada CNN dibuat secara paralel penuh, maka kebutuhan DSP dapat meningkat sangat besar [22]. Oleh karena itu, perancangan akselerator CNN pada FPGA perlu mempertimbangkan keseimbangan antara jumlah operasi paralel, penggunaan DSP, dan waktu komputasi.

2.2.4 Time Multiplexer

Time multiplexer adalah teknik penggunaan satu sumber daya hardware secara bergantian pada waktu yang berbeda. Dalam akselerator CNN, teknik ini digunakan ketika jumlah operasi MAC yang dibutuhkan lebih banyak dibandingkan jumlah DSP yang tersedia. Dengan teknik ini, satu unit MAC tidak hanya digunakan untuk satu operasi saja, tetapi dapat dipakai berulang untuk memproses beberapa data, channel, filter, atau neuron secara berurutan.

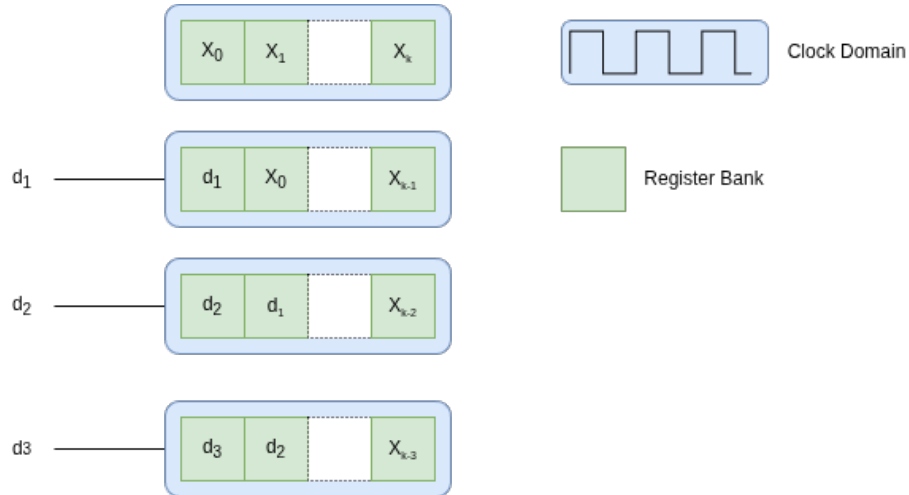
Berbeda dengan paralel penuh yang menyediakan banyak unit MAC sekaligus, time multiplexer menggunakan pendekatan berbasis pembagian waktu. Setiap siklus clock atau beberapa siklus clock digunakan untuk memproses bagian data tertentu, kemudian unit komputasi yang sama digunakan kembali untuk bagian data berikutnya [23, 14, 24]. Pendekatan ini dapat mengurangi penggunaan resource FPGA, tetapi waktu komputasi dapat menjadi lebih panjang karena sebagian operasi dilakukan secara bergantian.

Time multiplexing berkaitan erat dengan keterbatasan resource seperti DSP, LUT, dan memori internal [25]. Beberapa penelitian akselerator CNN pada FPGA menggunakan pendekatan *resource multiplexing* untuk mengurangi konsumsi hardware dengan cara memakai kembali unit komputasi yang sama pada beberapa tahap proses [22]. Pendekatan serupa juga digunakan pada arsitektur akselerator neural network yang menyesuaikan jumlah resource komputasi terhadap kebutuhan throughput tiap layer [24].

2.2.5 Shift Register

Shift register adalah rangkaian register yang digunakan untuk menyimpan dan menggeser data dari satu posisi ke posisi berikutnya pada setiap siklus *clock*. Pada sistem digital, shift register banyak digunakan ketika data harus diproses secara berurutan, terutama pada aliran data yang masuk secara *stream*. Setiap data baru yang

masuk akan mendorong data sebelumnya ke posisi berikutnya, sehingga beberapa data terakhir dapat tetap tersedia secara bersamaan di dalam rangkaian register.



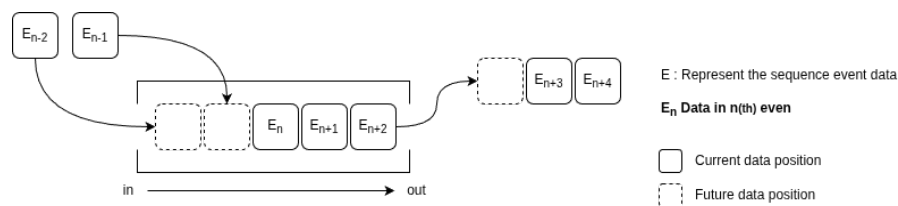
Gambar 2.2: Ilustrasi data *input shift register*.

Pada operasi konvolusi citra, shift register memiliki peran penting karena kernel tidak hanya membutuhkan satu piksel, tetapi juga piksel-piksel tetangga di sekitarnya. Misalnya pada kernel berukuran 3×3 , proses konvolusi membutuhkan sembilan nilai piksel yang berasal dari tiga baris dan tiga kolom yang berdekatan [2]. Jika data citra masuk secara berurutan dari kiri ke kanan dan dari atas ke bawah, maka sistem tidak dapat langsung memperoleh sembilan piksel tersebut tanpa mekanisme penyimpanan sementara. Oleh karena itu, shift register digunakan untuk mempertahankan beberapa piksel terakhir dalam satu baris, sehingga susunan piksel horizontal dapat dibentuk secara bertahap.

Dalam akselerator CNN berbasis FPGA, shift register membantu membentuk *window* konvolusi secara langsung dari aliran data. Setelah data piksel masuk ke dalam jalur pemrosesan, nilai-nilai piksel tersebut digeser pada setiap siklus *clock* hingga membentuk susunan data yang sesuai dengan ukuran kernel. Dengan cara ini, operasi *multiply accumulate* pada kernel dapat dilakukan tanpa harus membaca ulang seluruh data citra dari memori utama. Pendekatan seperti ini sejalan dengan implementasi CNN pada FPGA yang menekankan penggunaan *pipeline*, komputasi *fixed-point*, serta pengurangan akses memori eksternal untuk meningkatkan efisiensi sistem [11, 23].

2.2.6 First In First Out (FIFO)

First In First Out atau FIFO adalah struktur penyimpanan data sementara yang bekerja dengan prinsip data yang pertama masuk akan menjadi data pertama yang keluar. FIFO banyak digunakan pada sistem digital untuk mengatur aliran data antarblok, terutama ketika dua bagian sistem tidak selalu bekerja pada kecepatan yang sama [26]. Dalam implementasi FPGA, FIFO dapat digunakan sebagai penyangga data agar proses pengiriman dan penerimaan data tetap stabil meskipun terdapat perbedaan waktu proses antarblok.



Gambar 2.3: Ilustrasi FIFO

Pada arsitektur konvolusi, FIFO dapat digunakan sebagai *line buffer* untuk menyimpan satu atau beberapa baris data citra [11]. Ketika piksel citra masuk secara berurutan, FIFO menahan data dari baris sebelumnya sehingga data tersebut masih tersedia ketika baris berikutnya sedang diproses. Untuk membentuk *window* 3×3 , sistem umumnya membutuhkan dua FIFO sebagai penyangga dua baris sebelumnya. Data dari baris saat ini, baris sebelumnya, dan dua baris sebelumnya kemudian digabungkan dengan shift register untuk menghasilkan sembilan piksel yang dibutuhkan oleh kernel konvolusi.

Hubungan antara FIFO dan shift register dapat dilihat sebagai dua tingkatan penyimpanan yang saling melengkapi. FIFO bekerja pada tingkat baris citra, sedangkan shift register bekerja pada tingkat piksel dalam satu baris [11]. FIFO menyediakan data vertikal dari baris yang berbeda, sedangkan shift register menyusun data horizontal dari beberapa piksel yang berurutan. Dengan kombinasi ini, sistem dapat membentuk *sliding window* tanpa harus mengambil data berulang kali dari memori utama. Hal ini penting karena operasi konvolusi pada CNN membutuhkan akses data yang intensif, sehingga efisiensi penyimpanan sementara dan penggunaan ulang data menjadi bagian penting dalam rancangan akselerator CNN berbasis FPGA [23].

2.2.7 Kuantitasi

Kuantitasi merupakan proses mengubah representasi numerik dari presisi tinggi menjadi presisi yang lebih rendah. Pada CNN, parameter seperti bobot, bias, dan aktivasi pada awalnya direpresentasikan dalam format *floating-point* karena format tersebut fleksibel untuk proses pelatihan [23]. Namun, format *floating-point* tidak efisien untuk implementasi perangkat keras karena membutuhkan rangkaian aritmetika yang lebih kompleks, ukuran data yang lebih besar, serta konsumsi sumber daya yang lebih tinggi. Oleh karena itu, kuantitasi menjadi tahap penting untuk mengubah model CNN agar lebih sesuai dengan karakteristik FPGA.

Pada implementasi CNN berbasis FPGA, kuantitasi berperan langsung terhadap efisiensi memori, bandwidth, jumlah sumber daya aritmetika, dan latensi inferensi. Jacob et al. menunjukkan bahwa inferensi CNN dapat dijalankan dengan aritmetika integer melalui skema kuantitasi yang tetap menjaga akurasi model [27]. Gupta et al. menunjukkan bahwa jaringan saraf dapat bekerja menggunakan representasi *fixed-point* 16-bit dengan degradasi akurasi yang kecil ketika proses pembulatan dilakukan secara tepat [28]. Lin et al. kemudian membahas kuantitasi *fixed-point* pada jaringan konvolusional dengan menekankan pemilihan *bit-width* yang sesuai pada setiap layer [29]. Pada konteks FPGA, Goyal et al. dan Solovyev et al. memperkuat bahwa perubahan dari *floating-point* menuju *fixed-point* dapat menurunkan kebutuhan komputasi dan memori sehingga model lebih layak diimplementasikan pada platform embedded dan akselerator perangkat keras [30, 11]. Dengan demikian, kuantitasi tidak hanya dilihat sebagai proses kompresi model, tetapi sebagai bagian dari strategi perancangan arsitektur agar operasi CNN dapat dipetakan secara efisien ke dalam jalur data FPGA.

Kuantitasi *int8* berfokus pada pemetaan nilai real ke bilangan integer 8-bit menggunakan faktor skala dan *zero-point*. Jacob et al. menggunakan skema ini untuk mendukung inferensi berbasis aritmetika integer, sedangkan Krishnamoorthi menekankan bahwa kuantitasi 8-bit pada bobot dan aktivasi dapat menurunkan ukuran model serta mempercepat inferensi pada perangkat yang mendukung operasi integer [27, 31].

$$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}} \quad (2.2)$$

$$Z = \text{round} \left(q_{\min} - \frac{x_{\min}}{S} \right) \quad (2.3)$$

$$q_8 = \text{clip} \left(\text{round} \left(\frac{x}{S} \right) + Z, q_{\min}, q_{\max} \right) \quad (2.4)$$

$$\hat{x}_8 = S(q_8 - Z) \quad (2.5)$$

Kuantitasi *fixed-point* 16-bit menggunakan posisi bit pecahan yang tetap, sehingga nilai real dikonversi menjadi bilangan integer bertanda dengan faktor skala 2^F . Gupta et al. membuktikan bahwa representasi 16-bit *fixed-point* dapat digunakan pada jaringan saraf dengan hasil yang tetap kompetitif, sedangkan Lin et al. dan Goyal et al. menunjukkan pentingnya pemilihan jumlah bit dan posisi pecahan agar kesalahan kuantitasi tidak merusak akurasi model [28, 29, 30]. Pada format ini, F menyatakan jumlah bit pecahan yang digunakan. Proses kuantitasi dan pendekatan kembali ke nilai real dapat dituliskan sebagai berikut.

$$q_{16} = \text{clip} \left(\text{round}(x \cdot 2^F), -2^{15}, 2^{15} - 1 \right) \quad (2.6)$$

$$\hat{x}_{16} = q_{16} \cdot 2^{-F} \quad (2.7)$$

Untuk parameter CNN, bobot, aktivasi, dan bias dapat dikonversi menggunakan jumlah bit pecahan yang disesuaikan dengan kebutuhan masing-masing parameter.

$$q_w = \text{clip} \left(\text{round}(w \cdot 2^{F_w}), -2^{15}, 2^{15} - 1 \right) \quad (2.8)$$

$$q_a = \text{clip} \left(\text{round}(a \cdot 2^{F_a}), -2^{15}, 2^{15} - 1 \right) \quad (2.9)$$

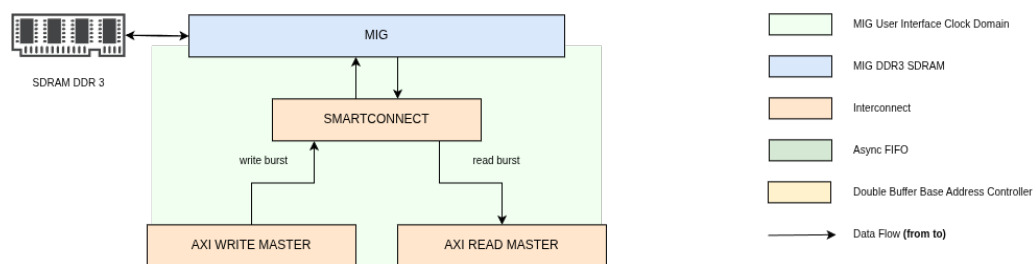
$$q_b = \text{clip} \left(\text{round}(b \cdot 2^{F_b}), -2^{31}, 2^{31} - 1 \right) \quad (2.10)$$

Trade-off utama antara *int8* dan *fixed-point* 16-bit terletak pada keseimbangan antara efisiensi dan kestabilan numerik. *Int8* memberikan ukuran data yang lebih kecil, kebutuhan memori yang lebih rendah, dan potensi bandwidth yang lebih hemat, tetapi membutuhkan pemilihan skala, *zero-point*, dan proses kalibrasi yang lebih hati-hati agar akurasi tidak turun secara signifikan. Sebaliknya, *fixed-point* 16-bit membutuhkan sumber daya yang lebih besar dibandingkan *int8*, tetapi memberikan rentang dan ketelitian yang lebih aman untuk menjaga hasil komputasi tetap dekat dengan model *floating-point*. Pada penelitian ini, *fixed-point* 16-bit dipilih karena memberik-

an titik tengah yang kuat antara efisiensi implementasi FPGA dan kestabilan akurasi inferensi.

2.2.8 Double Data Rate (DDR)

Double Data Rate atau DDR merupakan memori eksternal yang digunakan untuk menyediakan kapasitas penyimpanan lebih besar dibandingkan memori internal FPGA. Pada sistem berbasis citra, kebutuhan memori menjadi penting karena data yang diproses tidak hanya berupa satu piksel, tetapi dapat berupa satu frame penuh atau blok data berukuran besar. Untuk citra RGB565 beresolusi 640×480 , satu frame membutuhkan kapasitas penyimpanan yang jauh lebih besar dibandingkan penyimpanan lokal yang idealnya digunakan untuk buffer kecil, register, atau blok komputasi. Oleh karena itu, DDR ditempatkan sebagai *frame buffer* utama, sedangkan BRAM digunakan untuk menyimpan data sementara pada jalur pemrosesan yang lebih dekat dengan logika FPGA. Gambar 2.4 menunjukkan posisi DDR pada sistem kamera, AXI, MIG, dan VGA.



Gambar 2.4: Ilustrasi hubungan AXI, MIG, dan DDR pada sistem pemindahan data berbasis memori eksternal.

Advanced eXtensible Interface atau AXI merupakan protokol antarmuka yang digunakan untuk menghubungkan modul komputasi dengan subsistem memori atau periferan pada sistem digital. AXI mendukung transaksi *read* dan *write* secara terpisah, memiliki kanal alamat, data, dan respons, serta memungkinkan transfer data dalam bentuk *burst*. Karakter ini membuat AXI sesuai untuk pemindahan data berukuran besar karena beberapa data dapat dikirim atau diambil secara berurutan tanpa harus membentuk alamat baru untuk setiap data. Pada sistem pemrosesan citra, AXI dapat digunakan untuk mengirim data ke memori melalui AXI *write* master dan mengambil kembali data dari memori melalui AXI *read* master. Jika diperlukan, buffer lokal seperti FIFO atau BRAM dapat ditempatkan di sisi modul untuk menye-

suaikan laju data, tetapi mekanisme utama pemindahan data tetap dilakukan melalui transaksi AXI [32].

Memory Interface Generator atau MIG merupakan IP yang digunakan sebagai pengendali akses DDR pada FPGA. DDR tidak dapat diakses secara langsung seperti register biasa karena memori ini memiliki aturan waktu, proses inisialisasi, kalibrasi, pengaturan alamat, dan sinyal fisik yang lebih kompleks. MIG menyederhanakan bagian tersebut dengan menyediakan antarmuka yang dapat digunakan oleh logika FPGA, salah satunya melalui AXI. Dengan adanya MIG, modul logika tidak perlu mengatur sinyal fisik DDR secara langsung. Modul cukup membentuk transaksi baca atau tulis melalui antarmuka AXI, sedangkan MIG menerjemahkan transaksi tersebut menjadi operasi memori yang sesuai dengan protokol DDR. Peran ini membuat MIG menjadi penghubung penting antara logika digital di dalam FPGA dan memori eksternal yang memiliki aturan akses lebih ketat [33].

Penggunaan DDR pada akselerator CNN dilakukan pada penelitian yang membutuhkan penyimpanan data berukuran besar dan akses memori yang terstruktur. Solovyev et al. menggunakan memori eksternal sebagai bagian dari sistem pemrosesan video real-time, sementara memori lokal tetap digunakan untuk mendukung aliran data yang dekat dengan blok komputasi [11]. Zhang et al. menggunakan DDR3 sebagai penyimpanan data utama pada akselerator CNN berbasis FPGA, kemudian mengoptimalkan akses data menggunakan *loop tiling* dan pemanfaatan BRAM agar bandwidth memori dapat digunakan secara lebih efisien [23]. Kedua penelitian tersebut menunjukkan bahwa DDR berperan sebagai penyimpanan eksternal ketika ukuran data melebihi kapasitas memori lokal FPGA, sedangkan memori internal digunakan untuk mempercepat akses data yang sedang aktif diproses.

2.2.9 FPGA ARTIX 7

FPGA Artix-7 merupakan keluarga FPGA dari Xilinx yang dirancang untuk memberikan keseimbangan antara performa, konsumsi daya, dan biaya implementasi. Pada penelitian ini, board yang digunakan adalah Arty A7-100T dengan perangkat FPGA XC7A100T. Board ini menyediakan sumber daya logika, memori internal, DSP, dan DDR eksternal yang cukup untuk membangun sistem akselerator CNN skala embedded. Spesifikasi utama Arty A7-100T ditunjukkan pada Tabel 2.1.

Tabel 2.1: Spesifikasi utama Arty A7-100T

Komponen	Spesifikasi
FPGA	Xilinx Artix-7 XC7A100T
LUT	63.400
Flip-Flop	126.800
Block RAM	4.860 Kb
DSP Slice	240 DSP48E1
DDR	256 MB DDR3L

Jika dibandingkan dengan penelitian sebelumnya, Arty A7-100T berada pada kelas menengah. Board ini memiliki sumber daya lebih besar dibandingkan FPGA lainnya seperti Spartan-6 atau Cyclone IV yang digunakan pada penelitian Solovyev et al., tetapi masih lebih terbatas dibandingkan board kelas tinggi seperti Virtex-7 VC707 yang digunakan pada penelitian Zhang et al. [11, 23]. Kondisi ini membuat Arty A7-100T cukup mampu untuk implementasi CNN *accelerator*, karena desain harus benar-benar memperhatikan pemakaian resource. Operasi MAC dapat dipetakan ke DSP48E1, data sementara dapat disimpan pada BRAM, sedangkan FF dan LUT digunakan untuk membangun jalur kendali, pipeline, FSM, serta register antarblok.

Dari sisi daya dan latensi, FPGA Artix-7 mendukung pendekatan yang lebih hemat daya dibandingkan komputasi umum karena jalur data dapat dibuat khusus sesuai kebutuhan algoritma. Penggunaan komputasi *fixed-point*, DSP untuk operasi MAC, serta pipeline dapat menurunkan latensi komputasi karena operasi tidak perlu dijalankan sebagai instruksi perangkat lunak satu per satu. Namun, performa akhir tetap bergantung pada keseimbangan antara jumlah operasi paralel, akses DDR, dan penggunaan *time multiplexing*. Semakin banyak operasi dibuat paralel, latensi dapat ditekan, tetapi konsumsi resource dan daya dinamis meningkat. Sebaliknya, pemakaian ulang resource dapat menghemat DSP dan LUT, tetapi waktu komputasi menjadi lebih panjang.

2.2.10 OV7670

OV7670 merupakan sensor kamera CMOS beresolusi VGA yang dapat menghasilkan citra hingga ukuran 640×480 piksel. Sensor ini banyak digunakan pada implementasi FPGA karena menyediakan keluaran data paralel dan sinyal sinkronisasi yang relatif mudah dihubungkan dengan logika digital. Konfigurasi internal kamera

dilakukan melalui Serial Camera Control Bus atau SCCB, sedangkan data citra dikeluarkan melalui jalur data paralel yang disertai sinyal *pixel clock*, HREF, dan VSYNC [12].

Pada aliran data kamera, sinyal VSYNC digunakan sebagai penanda pergantian frame, HREF digunakan untuk menunjukkan bahwa data pada satu baris sedang valid, dan *pixel clock* digunakan sebagai acuan pengambilan data piksel. OV7670 dapat menghasilkan beberapa format keluaran, seperti YUV, RGB, dan RGB565. Format RGB565 sering digunakan pada sistem FPGA karena satu piksel dapat direpresentasikan dalam 16-bit, yaitu 5-bit merah, 6-bit hijau, dan 5-bit biru. Format ini cukup ringan untuk disimpan dan diproses, tetapi tetap mampu menampilkan warna yang memadai untuk sistem embedded vision.

Dalam sistem CNN berbasis kamera, OV7670 berperan sebagai sumber data visual yang masuk ke sistem secara kontinu. Hal ini membuat logika penerima kamera harus mampu membaca data berdasarkan sinyal sinkronisasi, menyusun data piksel dengan benar, dan menjaga agar aliran data tidak terputus. Karena data kamera datang mengikuti domain *pixel clock*, rancangan FPGA perlu memperhatikan sinkronisasi, buffering, dan validitas data sebelum data tersebut digunakan untuk pemrosesan atau ditampilkan kembali.

2.2.11 VGA

Video Graphics Array atau VGA merupakan antarmuka tampilan analog yang mengirimkan sinyal warna merah, hijau, dan biru bersama sinyal sinkronisasi horizontal dan vertikal. Pada FPGA, tampilan VGA dibentuk dengan menghasilkan data warna RGB dan sinyal timing yang sesuai dengan resolusi layar. Untuk resolusi 640×480 , sistem perlu menghasilkan urutan piksel, periode *blanking*, sinyal HSYNC, dan sinyal VSYNC agar monitor dapat menampilkan gambar dengan stabil.

Pada penelitian ini, keluaran VGA dapat dibantu menggunakan Pmod VGA dari Digilent. Pmod VGA menyediakan jalur warna 12-bit, yaitu 4-bit untuk merah, 4-bit untuk hijau, dan 4-bit untuk biru, serta dua sinyal sinkronisasi standar berupa HSYNC dan VSYNC [34]. Modul ini mempermudah FPGA untuk menghasilkan keluaran VGA karena konversi sinyal digital RGB menuju level analog dilakukan melalui rangkaian resistor pada Pmod. Dengan demikian, FPGA cukup menghasilkan data warna digital dan sinyal sinkronisasi yang sesuai dengan timing VGA.

Dalam sistem embedded vision, VGA berfungsi sebagai jalur keluaran visual

untuk menampilkan hasil pembacaan kamera atau hasil pemrosesan citra. Keberadaan VGA membuat sistem dapat diamati secara langsung tanpa harus selalu mengirim data ke komputer. Dari sisi perancangan, bagian VGA menuntut aliran data yang stabil karena tampilan layar bergerak mengikuti timing yang tetap. Jika data piksel tidak tersedia pada saat dibutuhkan, tampilan dapat mengalami gangguan seperti warna tidak sesuai, gambar bergeser, atau frame terlihat tidak stabil.

2.3 Analisis Perbandingan Metode

Tabel 2.2: Perbandingan implementasi sistem pada penelitian terkait

Penelitian	CNN	Kamera	VGA	Board/Platform
Solovyev et al. [11]	Ya	Ya	Ya	<i>DE0-Nano & Intel Cyclone IV</i>
Qiu et al. [13]	Ya	–	–	<i>Xilinx ZC706</i>
Dua et al. [14]	Ya	–	–	<i>Arria 10 GX1150 & Stratix 10 GX2800</i>
Navaneethan et al. [15]	–	Ya	Ya	<i>Spartan-6 & Artix-7</i>
Sun et al. [16]	–	Ya	Ya	<i>FPGA MicroBlaze based</i>

Berdasarkan Tabel 2.2, penelitian terkait dapat dikelompokkan menjadi dua arah utama. Penelitian seperti Solovyev et al. sudah menunjukkan implementasi inferensi CNN secara *end-to-end* pada FPGA, sedangkan Qiu et al. dan Dua et al. lebih menekankan optimasi akselerator CNN dari sisi komputasi model, arsitektur paralel, dan efisiensi operasi. Di sisi lain, Navaneethan et al. dan Sun et al. lebih berfokus pada integrasi sistem citra, yaitu pengambilan data dari kamera OV7670 dan penampilan citra melalui VGA, tetapi belum membahas akselerasi inferensi CNN. Hal ini menunjukkan adanya celah penelitian pada integrasi antara sistem akuisisi citra, tampilan video, dan akselerator CNN dalam satu platform FPGA. Oleh karena itu, penelitian ini tidak hanya berfokus pada implementasi CNN berbasis *fixed-point*, tetapi juga pada bagaimana sistem tersebut dapat diintegrasikan dengan periferal citra agar lebih mendekati kebutuhan aplikasi *embedded vision* secara langsung.

Tabel 2.3: Perbandingan performa penelitian terkait

Penelitian	Latensi/FPS	Arsitektur Model	Daya (W)	GOPS	Loss	Akurasi
[11]	>150 FPS	CNN	–	–	–	Tidak berubah setelah kuantisasi
[13]	224.60 ms	<i>VGG16-SVD</i>	9.63	137	–	86.66%
[14]	140ms	<i>AlexNet</i>	2,1	–	–	–

Selain aspek integrasi sistem, perbandingan performa pada Tabel 2.3 juga menunjukkan bahwa konsumsi daya menjadi faktor penting dalam implementasi CNN.

Pada penelitian Qiu et al., implementasi VGG16-SVD pada FPGA hanya membutuhkan daya sebesar 9,63 W, sedangkan platform CPU dan GPU yang digunakan sebagai pembanding memiliki konsumsi daya yang jauh lebih tinggi, yaitu 135 W untuk CPU dan 250 W untuk GPU [13]. Meskipun GPU mampu memberikan performa komputasi yang lebih besar, konsumsi dayanya mencapai sekitar 26 kali lebih tinggi dibandingkan FPGA. Kondisi ini memperlihatkan bahwa CPU dan GPU tidak selalu menjadi pilihan paling efisien untuk sistem *real-time* dan *embedded*, terutama ketika batasan daya dan latensi menjadi pertimbangan utama. Dengan demikian, FPGA menjadi alternatif yang relevan karena dapat menyediakan jalur komputasi khusus melalui paralelisme, *pipeline*, dan representasi *fixed-point* sehingga proses inferensi dapat dilakukan dengan konsumsi daya yang lebih rendah.

DAFTAR PUSTAKA

- [1] O. Elharrouss, Y. Akbari, N. Almaadeed, and S. A. Al-Maadeed, “Backbones-review: Feature extraction networks for deep learning and deep reinforcement learning approaches,” *ArXiv*, vol. abs/2206.08016, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:249712263>
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: <https://neurips.cc>
- [3] X. Liao, Y. Li, S. Zhang, X. Wei, and J. Hu, “Predicting gpu training energy consumption in data centers using task metadata via symbolic regression,” *Energies*, vol. 19, no. 2, 2026. [Online]. Available: <https://www.mdpi.com/1996-1073/19/2/448>
- [4] S. Stirenko, Y. Rosenberg, Y. Gordienko, O. Alienin, O. Rokovyi, and N. Alienina, “Batch size influence on performance of graphic and tensor processing units during training and inference phases,” *arXiv preprint arXiv:1812.11731*, pp. 1–9, 2018. [Online]. Available: <https://arxiv.org/abs/1812.11731>
- [5] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, “A survey of deep learning techniques for autonomous driving,” *Journal of Field Robotics*, vol. 37, no. 3, pp. 362–386, 2020.
- [6] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, “A survey of fpga-based neural network inference accelerators,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 12, no. 1, 2019.
- [7] K. Abdelouahab, M. Pelcat, J. Serot, and F. Berry, “Accelerating cnn inference on fpgas: A survey,” *arXiv preprint arXiv:1806.01683*, 2018.
- [8] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, “A performance study of general-purpose applications on graphics processors,” *Journal of Parallel and Distributed Computing*, vol. 69, no. 10, pp. 1370–1380, 2009.

- [9] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, M. Mattavelli, and J. Silas, "Comparing energy efficiency of cpu, gpu and fpga implementations for vision kernels," in *2019 IEEE International Conference on Embedded Software and Systems (ICCESS)*. IEEE, 2019, pp. 1–8.
- [10] Xilinx Inc., "Lowering power at 28 nm with xilinx 7 series fpgas," White Paper: WP389 (v1.2), 2015.
- [11] R. Solovyev, A. Kustov, D. Telpukhov, V. Rukhlov, and A. Kalinin, "Fixed-point convolutional neural network for real-time video processing in FPGA," in *2019 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIconRus)*. IEEE, 2019, pp. 1605–1611. [Online]. Available: <https://doi.org/10.1109/EIconRus.2019.8656778>
- [12] *OV7670/OV7171 CMOS VGA CameraChip Sensor Datasheet*, OmniVision Technologies, 2006, accessed: 2026-06-15. [Online]. Available: https://web.mit.edu/6.111/www/f2016/tools/OV7670_2006.pdf
- [13] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song, Y. Wang, and H. Yang, "Going deeper with embedded FPGA platform for convolutional neural network," in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '16. Association for Computing Machinery, 2016, pp. 26–35. [Online]. Available: <https://doi.org/10.1145/2847263.2847265>
- [14] A. Dua, Y. Li, and F. Ren, "Systolic-CNN: An OpenCL-defined scalable run-time-flexible FPGA accelerator architecture for accelerating convolutional neural network inference in cloud/edge computing," 2020. [Online]. Available: <https://arxiv.org/abs/2012.03177>
- [15] S. Navaneethan, C. Thanusha, M. A. Varshini, and A. Anil, "A hardware efficient real-time video processing on FPGA with OV7670 camera interface and VGA," in *2022 6th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*. IEEE, 2022, pp. 348–352. [Online]. Available: <https://doi.org/10.1109/ICECA55336.2022.10009129>
- [16] L. Sun, E. Sun, and Y. Yu, "Real time controllable multi channel HD digital video processing system based on FPGA and MicroBlaze," in *Proceedings of*

- the 2021 7th International Conference on Computing and Artificial Intelligence*, ser. ICCAI '21. Association for Computing Machinery, 2021, pp. 183–190. [Online]. Available: <https://doi.org/10.1145/3467707.3467734>
- [17] A. Cosoroaba, “Achieving high performance DDR3 data rates,” Xilinx, White Paper WP383, Aug. 2013. [Online]. Available: <https://docs.amd.com/api/khub/documents/SKHv3pFeACeh9A7QJryuMg/content>
- [18] A. Jacobo, “FPGA_OV7670_Camera_Interface: Real-time streaming of OV7670 camera via VGA,” GitHub repository, 2021, accessed: 2026-06-15. [Online]. Available: https://github.com/AngeloJacob/FPGA_OV7670_Camera_Interface
- [19] Arm Limited, *AMBA AXI and ACE Protocol Specification*, Arm Limited, 2020, non-Confidential. [Online]. Available: <https://developer.arm.com/documentation/ihi0022/latest>
- [20] —, *Introduction to AMBA AXI*, Arm Limited, 2020. [Online]. Available: <https://developer.arm.com/documentation/102202/latest>
- [21] Xilinx, *7 Series DSP48E1 Slice User Guide*, Xilinx, 2018. [Online]. Available: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1
- [22] F. Yan, F. Wang, C. Liu, Y. Wang, and J. Zhou, “Design of convolutional neural network processor based on fpga resource multiplexing architecture,” *Sensors*, vol. 22, no. 16, p. 5967, 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/16/5967>
- [23] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, “Optimizing FPGA-based accelerator design for deep convolutional neural networks,” in *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '15. Association for Computing Machinery, 2015, pp. 161–170. [Online]. Available: <https://doi.org/10.1145/2684746.2689060>
- [24] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “FINN: A framework for fast, scalable binarized neural network inference,” in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '17.

- Association for Computing Machinery, 2017, pp. 65–74. [Online]. Available: <https://doi.org/10.1145/3020078.3021744>
- [25] AMD, “68595 - DSP slice - using time division multiplexing or overclocking the DSP slices in Xilinx devices can greatly increase throughput and efficiency,” https://adaptivesupport.amd.com/s/article/68595?language=en_US, Feb. 2023, diakses pada: 16 Juni 2026.
- [26] C. E. Cummings and P. Alfke, “Simulation and synthesis techniques for asynchronous fifo design,” in *SNUG San Jose*. Sunburst Design, 2002. [Online]. Available: https://www.sunburst-design.com/papers/CummingsSNUG2002SJ_FIFO1.pdf
- [27] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
- [28] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, “Deep learning with limited numerical precision,” in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 1737–1746. [Online]. Available: <https://proceedings.mlr.press/v37/gupta15.html>
- [29] D. Lin, S. Talathi, and S. Annapureddy, “Fixed point quantization of deep convolutional networks,” in *Proceedings of the 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 48. PMLR, 2016, pp. 2849–2858. [Online]. Available: <https://proceedings.mlr.press/v48/linb16.html>
- [30] R. Goyal, J. Vanschoren, V. van Acht, and S. Nijssen, “Fixed-point quantization of convolutional neural networks for quantized inference on embedded platforms,” *arXiv preprint arXiv:2102.02147*, 2021. [Online]. Available: <https://arxiv.org/abs/2102.02147>

- [31] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” *arXiv preprint arXiv:1806.08342*, 2018. [Online]. Available: <https://arxiv.org/abs/1806.08342>
- [32] Arm Ltd., *AMBA AXI and ACE Protocol Specification*, Arm Ltd., Dec. 2017. [Online]. Available: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/IHI0022H_amba_axi_protocol_spec.pdf
- [33] Xilinx, *7 Series FPGAs Memory Interface Solutions User Guide*, Xilinx, Apr. 2012. [Online]. Available: https://www.xilinx.com/support/documents/ip_documentation/mig_7series/v4_1/ug586_7Series_MIS.pdf
- [34] Digilent, *VGA Display Controller*, Digilent Inc., 2024. [Online]. Available: <https://digilent.com/reference/learn/programmable-logic/tutorials/vga-display-controller/start>